

Nombres: Richard Steven

Apellidos: Arizandieta Yol

Carné: 202400111

Curso: (LAB) Introducción a la Programación y Computación 1

Catedrático: Neftalí Calderón

Auxiliar: Erwin Vásquez



MANUAL TÉCNICO PARA MANIPULACIÓN DE PROYECTO NO. 01

INVENTARIO Y TIENDA DEL DOJO COBRA KAI

GUATEMALA 09 DE DICIEMBRE DE 2024

1. Introducción

Este sistema es un programa en Java diseñado para gestionar inventarios, realizar ventas y generar reportes detallados. Incluye funcionalidades para gestionar productos, realizar ventas con facturación en HTML, y generar reportes como el "Top 5 de productos más vendidos". Este manual proporciona la guía técnica para comprender, modificar y mantener el código.

2. Requisitos del Sistema

2.1 Software Requerido

- **Java Development Kit (JDK):** Versión 8 o superior.
- **Editor o IDE:** Se recomienda Visual Studio Code, IntelliJ IDEA o Eclipse.
- **Sistema Operativo:** Windows, macOS o Linux.

2.2 Librerías Usadas

- `java.io`: Para manejo de archivos y generación de HTML.
- `java.util`: Para manejar entrada de datos con Scanner.

2.3 Estructura del Proyecto

El proyecto contiene:

- `Main.java`: Código principal.
- `src/Facturas`: Carpeta donde se almacenan las facturas generadas.
- `src/style.css`: Archivo CSS para dar formato a las facturas HTML.

3. Descripción Técnica

Librerías utilizadas

1. Propósito de las librerías importadas

1. `java.io.IOException`
 - **Propósito:** Manejar excepciones relacionadas con errores de entrada/salida (I/O).

- **Uso en el programa:** Se utiliza principalmente en la generación de facturas HTML o al procesar archivos en la carga masiva.

2. `java.io.PrintWriter`

- **Propósito:** Proporcionar una forma sencilla de escribir datos en archivos de texto, como facturas HTML.
- **Uso en el programa:** Para generar las facturas en formato HTML.

3. `java.util.Scanner`

- **Propósito:** Leer datos de entrada del usuario o archivos.
- **Uso en el programa:**
 - Para capturar datos del usuario (inicio de sesión, menú, ventas, etc.).
 - Para leer archivos durante la carga masiva.

4. `java.io.File`

- **Propósito:** Representar archivos y directorios en el sistema de archivos.
- **Uso en el programa:**
 - Verificar la existencia de archivos antes de procesarlos.
 - Crear carpetas (como Facturas) para almacenar las facturas generadas.

5. `java.io.FileNotFoundException`

- **Propósito:** Manejar la excepción cuando un archivo no se encuentra en la ruta especificada.
- **Uso en el programa:**
 - Capturar errores al intentar leer un archivo inexistente en la carga masiva.

```
//Librerias
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;
```

Inicio de Sesión

Esta sección implementa el proceso de **inicio de sesión**, asegurándose de que solo usuarios con credenciales válidas puedan acceder al sistema.

A. Declaración de variables

```
//VARIABLES DE INICIO DE SESIÓN
final String user = "sensei_202400111";
final String pass = "ipc1_202400111";
boolean acceso = false;
```

- **user y pass:**
 - Constantes (final) que almacenan el usuario y la contraseña predefinidos.
 - Son los valores contra los cuales se validará la entrada del usuario.
- **acceso:**
 - Controla el flujo del bucle do-while. Solo permite salir del bucle si las credenciales son correctas.

B. Validación de certificados

```

//Validando certificados
do{
    System.out.println("Ingrese su usuario: ");
    String tryuser = leer.nextLine();
    System.out.println("Ingrese su contraseña: ");
    String trypass = leer.nextLine();
    if(tryuser.equals(user) && trypass.equals(pass)){
        System.out.println("Acceso concedido, ingresando al sistema");
        System.out.println("-----");

        //Dar acceso al menú principal
        acceso = true;

    }else{
        System.out.println("Certificados incorrectos, inténtalo nuevamente");
        System.out.println("-----");
    }
}while(!acceso);

//LLEVAR AL USUARIO AL MENÚ PRINCIPAL
mainMenu(leer);

```

1. Solicita el usuario y contraseña al usuario.
2. **Valida las credenciales:**
 - Si coinciden con los valores predefinidos (user y pass), otorga acceso (acceso = true).
 - Si no coinciden, muestra un mensaje de error y permite reintentar.
3. **Bucle do-while:**
 - Garantiza que el usuario no pueda avanzar al menú principal hasta que se introduzcan credenciales válidas.
 - Una vez que el acceso es concedido, el control se transfiere al método mainMenu para que el usuario interactúe con las funcionalidades del sistema.

C. Validación de certificados

```

while(true){
    //BIENVENIDA
    System.out.println("\n--- Menú Principal ---");
    System.out.println("-----");
    System.out.println("1. Ingresar un nuevo producto (Individual)");
    System.out.println("2. Ingresar un lote de productos (Carga masiva)");
    System.out.println("3. Realizar una venta");
    System.out.println("4. Realizar un reporte");
    System.out.println("5. Salir del sistema");
    System.out.println("-----");
    System.out.println("Selecciona una opción para continuar");
}

```

- Presenta al usuario las opciones disponibles en el sistema.
- Utiliza números para identificar cada opción y facilitar la selección.

D. Elegir opción

```
int opcion = leer.nextInt();
leer.nextLine(); //Limpiar el buffer
//Selección de opciones
switch(opcion){
    case 1:
        agregarProducto(leer);
        break;
    case 2:
        agregarLote(leer);
        break;
    case 3:
        //COMPROBAMOS SI HAY PRODUCTOS EN EL INVENTARIO
        if(cantidadProducto == 0){
            System.out.println("No hay productos en el inventario, agrega antes de realizar una venta");
            System.out.println("Presiona cualquier tecla para regresar al menú principal...");
            leer.nextLine();
            break;
        }else {
            realizarVenta(leer);
        }
        break;
    case 4:
        realizarReporte(leer);
        break;
    case 5:
        System.out.println("Saliendo del sistema...");
        System.exit(status: 0);
        break;
    default:
        System.out.println("Opción invalida, intenta de nuevo");
        System.out.println("Presiona cualquier tecla para regresar al menú principal...");
        leer.nextLine();
        break;
}
```

- Captura la entrada del usuario mediante Scanner.
 - Utiliza nextInt() para obtener un número entero y nextLine() para limpiar el buffer del scanner.
1. **Opción 1** - Ingresar un nuevo producto:
 - Llama al método agregarProducto para añadir productos individualmente.

2. **Opción 2** - Ingresar un lote de productos:

- Llama al método agregarLote para realizar una carga masiva desde un archivo.

3. **Opción 3** - Realizar una venta:

- Verifica que haya productos disponibles en el inventario (cantidadProducto > 0).
- Si no hay productos, muestra un mensaje de advertencia.
- Si hay productos, llama al método realizarVenta.

4. **Opción 4** - Generar un reporte:

- Llama al método realizarReporte para acceder a las opciones de reportes.

5. **Opción 5** - Salir del sistema:

- Utiliza System.exit(0) para finalizar el programa.

Opción inválida (default):

- Informa al usuario que la opción seleccionada no es válida y solicita intentarlo nuevamente.

E. Gestión de Productos Individuales

El método agregarProducto permite añadir productos al inventario manualmente. El sistema valida si el producto ya existe, verifica que el precio sea válido, y lo almacena en los arreglos nombreProducto y precioProducto.

E.2 Variables Globales

```
//OPERACION PARA PRODUCTOS INDIVIDUALES
//CREAR ARRAYS PARA ALMACENAR NOMBRE Y PRECIO DE PRODUCTOS
static String[] nombreProducto = new String[100]; 13 usages
static double[] precioProducto = new double[100]; 5 usages
static int cantidadProducto = 0; 14 usages
```

1. **nombreProducto[]:**

- Arreglo para almacenar los nombres de los productos.
- Tamaño fijo de 100 elementos.

2. **precioProducto[]:**

- Arreglo para almacenar los precios correspondientes a cada producto.

3. **cantidadProducto:**

- Contador para llevar el número de productos actualmente registrados en el inventario.

E.3 Solicitar el Producto

```
//CREANDO METODO PARA AGREGAR PRODUCTOS
public static void agregarProducto(Scanner leer){ 1 usage
    System.out.println("\n--- Ingresar un nuevo producto (Individual) ---");
    System.out.println("-----");
    System.out.println("Ingresa el nombre del producto: ");
    String nombre = leer.nextLine();

    //Verificar si el producto ya existe
    for(int i = 0; i < cantidadProducto; i++){
        if(nombreProducto[i].equalsIgnoreCase(nombre)){
            System.out.println("El producto ya existe en el inventario");
            System.out.println("Presiona cualquier tecla para regresar al menú principal...");
            leer.nextLine();
            return;
        }
    }
}
```

Captura el nombre del producto ingresado por el usuario.

- Itera sobre el arreglo nombreProducto hasta cantidadProducto para verificar si el producto ya está registrado.
- Si se encuentra, se notifica al usuario y se termina la ejecución del método (return).

E.4 Validación: Producto duplicado

- Recorre el arreglo nombreProducto para verificar si el producto ya está registrado.
- Usa el método equalsIgnoreCase para comparar el nombre ingresado, ignorando diferencias entre mayúsculas y minúsculas.
- Si el producto ya existe, notifica al usuario y regresa al menú principal.

E.5 Captura del precio del producto

```
//Ingresar el precio del producto
System.out.println("Ingresa el precio del producto: ");
double precio = leer.nextDouble();
leer.nextLine(); //Limpiar el buffer

//Comprobar que el precio sea mayor a 0
if(precio <= 0){
    System.out.println("El precio del producto debe ser mayor a 0");
    System.out.println("Presiona cualquier tecla para regresar al menú principal...");
    leer.nextLine();
    return;
}

//Agregar el producto al inventario
if(cantidadProducto < nombreProducto.length){
    nombreProducto[cantidadProducto] = nombre;
    precioProducto[cantidadProducto] = precio;
    cantidadProducto++;
    System.out.println("Producto agregado correctamente");
    System.out.println("Presiona cualquier tecla para regresar al menú principal...");
    leer.nextLine();
}else{
    System.out.println("No se pueden agregar más productos, el inventario está lleno");
    System.out.println("Presiona cualquier tecla para regresar al menú principal...");
    leer.nextLine();
}
}
```

- Solicita al usuario el precio del producto y lo almacena en la variable precio.

E.6 Validación: Precio positivo

- Verifica que el precio ingresado sea mayor a 0.
- Si el precio es inválido, muestra un mensaje de error y regresa al menú principal.

F. Almacenamiento del producto

1. Verifica si hay espacio disponible en el inventario:

- Comprueba que cantidadProducto no exceda la capacidad del arreglo.

2. Registra el producto en el inventario:

- Almacena el nombre en nombreProducto y el precio en precioProducto.
- Incrementa cantidadProducto para reflejar el nuevo total.

3. Notifica el resultado al usuario:

- Muestra un mensaje de confirmación si el producto fue agregado exitosamente.
- Si el inventario está lleno, notifica que no se pueden agregar más productos.

Carga Masiva de Productos

```
//OPERACIÓN PARA CARGA MASIVA
public static void agregarLote(Scanner leer){ 1 usage
    System.out.println("\n--- Carga Masiva de Productos ---");
    System.out.println("-----");
    System.out.println("Ingresa la ruta del archivo");
    String rutaArchivo = leer.nextLine();

    File archivo = new File(rutaArchivo);

    //Comprobando si existe el archivo
    //Usando "!" para negar la preposición
    if(!archivo.exists()){
        System.out.println("El archivo no existe, comprueba la ruta e intenta de nuevo");
        System.out.println("Presiona cualquier tecla para continuar...");
        leer.nextLine();
        return;
    }
}
```

A. Solicitar la ruta del archivo

- **Función:** Solicita al usuario que ingrese la ruta completa del archivo que contiene los productos y precios.
- **Variable File:**
 - Representa el archivo especificado por la ruta.
 - Permite realizar operaciones como verificar si el archivo existe.

B. Verificar la existencia del archivo

- **Función:** Asegura que el archivo ingresado exista antes de intentar procesarlo.

- **Validación:**

- Si el archivo no existe (!archivo.exists()), muestra un mensaje de error.
- Solicita al usuario volver a intentarlo.
- Regresa al menú principal utilizando return.

Procesamiento del Archivo

```
try(Scanner escanearArchivo = new Scanner(archivo)){
    //Variables de productos
    int productosAgregados = 0;
    int productosDuplicados = 0;

    //Saltando la primer linea de encabezados
    if(escanearArchivo.hasNextLine()){
        escanearArchivo.nextLine();
    }

    //Leyendo el archivo
    while(escanearArchivo.hasNextLine()){
        String linea = escanearArchivo.nextLine();

        String [] partes = linea.split(regex: ",");

        //Comprobando que tengan el formato correcto
        if(partes.length != 2) {
            System.out.println("Error en el producto: " + linea);
            System.out.println("El formato correcto es: nombre;precio");
            productosDuplicados++;
            continue;
        }
    }
}
```

Estructura del código

A. Apertura del archivo

- **try con recursos:** Asegura que el archivo se cierre automáticamente después de ser procesado, evitando fugas de recursos.

B. Inicialización de variables

- **productosAgregados:** Contador para los productos válidos que se añaden al inventario.
- **productosDuplicados:** Contador para los productos que no cumplen con los criterios o están duplicados.

C. Saltar encabezados

- **Función:** Omite la primera línea del archivo, asumiendo que contiene encabezados (por ejemplo, "Producto;Precio").
- **hasNextLine():** Verifica si hay más líneas disponibles en el archivo antes de avanzar.

D. Lectura y validación de líneas

1. Leer una línea del archivo:

- Cada línea representa un producto en formato nombre;precio.

2. Dividir la línea en partes:

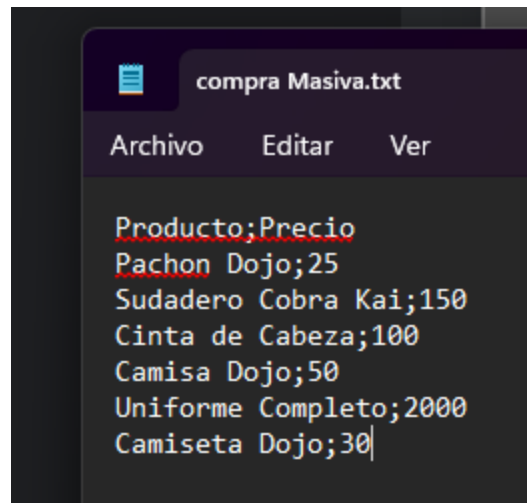
- **split(";"):** Divide la línea en un arreglo de dos partes (nombre y precio) utilizando el punto y coma como delimitador.

E. Validación del formato

1. Comprobar el formato de la línea:

- Valida que la línea contenga exactamente dos partes.
- Si no cumple:
 - Muestra un mensaje de error indicando el problema.
 - Incrementa el contador de productos duplicados.
 - Usa **continue** para pasar a la siguiente línea, ignorando la actual.

2. Formato esperado:



Validación de contenido

```
String nombre = partes[0].trim();
double precio = 0;

try{
    precio = Double.parseDouble(partes[1].trim());
}catch(NumberFormatException e){
    System.out.println("Precio invalido en el producto: " + linea);
    productosDuplicados++;
    continue;
}

//Verificando el precio del producto
if (precio <= 0){
    System.out.println("El precio del siguiente producto debe ser mayor a cero: " + linea);
    productosDuplicados++;
    continue;
}

//Verificar si el producto ya existe
if(productoExiste(nombre)) {
    System.out.println("El siguiente producto ya existe en el inventario: " + linea);
    productosDuplicados++;
    continue;
}
```

A. Captura del nombre

- partes[0]: Contiene el nombre del producto, obtenido del archivo.

- `trim()`: Elimina espacios en blanco al inicio y final del texto, asegurando un nombre limpio.

B. Conversión del precio

1. Convertir el precio a un número:

- `partes[1]`: Contiene el precio del producto, en formato de texto.
- `Double.parseDouble`: Convierte el precio a tipo `double`.
- `trim()`: Elimina espacios en blanco antes de la conversión.

2. Manejo de errores:

- Si el precio no es un número válido, lanza una excepción `NumberFormatException`.
- Captura la excepción y:
 - Muestra un mensaje indicando el error.
 - Incrementa el contador de productos duplicados (`productosDuplicados++`).
 - Usa `continue` para pasar a la siguiente línea, ignorando el producto actual.

C. Validación del precio

1. Garantizar que el precio sea mayor a cero:

- Si el precio es menor o igual a 0, el producto no es válido.

2. Acción ante error:

- Muestra un mensaje indicando el problema.
- Incrementa el contador de productos duplicados.
- Usa `continue` para ignorar el producto actual.

D. Verificación de duplicados

1. Revisar si el producto ya está registrado:

- Llama al método productoExiste para buscar el nombre del producto en el inventario actual.

2. Acción ante duplicado:

- Muestra un mensaje indicando que el producto ya existe.
- Incrementa el contador de productos duplicados.
- Usa continue para pasar a la siguiente línea.

Agregar productos al inventario

```
//Agregar el producto al inventario
if(cantidadProducto < nombreProducto.length){
    nombreProducto[cantidadProducto] = nombre;
    precioProducto[cantidadProducto] = precio;
    cantidadProducto++;
    productosAgregados++;
}else{
    System.out.println("No se pueden agregar más productos, el inventario está lleno.")
    productosDuplicados++;
}
}
} catch (FileNotFoundException e) {
    System.out.println("Error al abrir el archivo: " + e.getMessage());
}

System.out.println("Productos añadidos de manera exitosa!");
System.out.println("Presiona cualquier tecla para continuar...");
leer.nextLine();
```

A. Validación de capacidad del inventario

1. Verificar espacio disponible:

- Comprueba que el índice cantidadProducto no exceda la capacidad del arreglo (nombreProducto.length).
- **Condición:** Si aún hay espacio disponible, se permite agregar el producto.

2. Agregar el producto:

- Almacena el nombre y el precio en los arreglos `nombreProducto` y `precioProducto`, respectivamente.
- Incrementa el contador `cantidadProducto` para reflejar el nuevo total de productos registrados.
- Incrementa `productosAgregados` para llevar el conteo de los productos añadidos exitosamente.

3. **Acción si el inventario está lleno:**

- Si no hay espacio en el inventario, muestra un mensaje indicando que está lleno.
- Incrementa `productosDuplicados` porque el producto no pudo ser agregado.

B. Manejo de excepciones

- **Propósito:** Captura y maneja el error si el archivo no puede ser encontrado o abierto.
- **Acción:**
 - Muestra un mensaje de error con detalles específicos proporcionados por `e.getMessage()`.

C. Mensaje final y espera del usuario

1. **Confirmación de éxito:**

- Informa al usuario que el proceso de carga masiva ha finalizado.
- El mensaje es genérico y no indica el número de productos agregados o ignorados.

2. **Pausa antes de regresar al menú:**

- Usa `leer.nextLine()` para esperar la interacción del usuario antes de continuar con el flujo del programa.

Verificar si un producto existe

```
//Metodo para analizar si un producto existe
public static boolean productoExiste(String nombre){ 1 usage
    for(int i = 0; i < cantidadProducto; i++){
        if(nombreProducto[i].equalsIgnoreCase(nombre)){
            return true;
        }
    }
    return false;
}
```

A. Declaración del método

- **Modificador static:** Permite que el método sea llamado sin necesidad de instanciar la clase.
- **Retorno boolean:** Devuelve true si el producto existe, o false en caso contrario.
- **Parámetro nombre:** Recibe el nombre del producto a buscar en el inventario.

B. Búsqueda en el inventario

1. Recorrer el inventario:

- El bucle for itera desde 0 hasta cantidadProducto - 1, asegurándose de recorrer solo las posiciones ocupadas del arreglo nombreProducto.

2. Comparación de nombres:

- equalsIgnoreCase: Compara el nombre ingresado con los nombres del inventario ignorando mayúsculas y minúsculas.
- Si encuentra un producto con el mismo nombre, retorna true de inmediato.

C. Resultado si no encuentra el producto

- Si el bucle termina sin encontrar el producto, retorna false, indicando que el producto no está en el inventario.

Realizar una Venta

```
//OPERACIÓN PARA VENDER PRODUCTOS
//VARIABLE PARA DEFINIR EL NUMERO DE VENTA, NOS SERVIRÁ PARA GENERAR DISTINTAS FACTURAS EN DISTINTOS ARCHIVOS HTML
static int numeroVenta = 1; 2 usages

public static void realizarVenta(Scanner leer){ 1 usage
    System.out.println("\n--- Realizar una venta ---");
    System.out.println("-----");
    System.out.println("Ingresa el nombre del cliente (presiona enter para consumidor final");
    String nombreCliente = leer.nextLine();
    int nitCliente = 0;

    //Verificamos si el cliente puso su nombre
    if(nombreCliente.isEmpty()){
        nombreCliente = "C/F";
    }else{
        System.out.println("Ingresa el NIT del cliente: ");
        nitCliente = leer.nextInt();
        leer.nextLine(); //Limpiar buffer
    }
}
```

A. Variable de control: Número de venta

- **Propósito:** Lleva un conteo de las ventas realizadas para asignar un número único a cada factura generada.
- Este valor se incrementa al final del método, garantizando que las facturas no se sobrescriban.

B. Mensaje de introducción

- Informa al usuario que está ingresando al flujo de ventas.
- Mejora la experiencia del usuario al mantener un diseño consistente.

C. Captura de datos del cliente

- nombreCliente:
 - Solicita el nombre del cliente.
 - Si el usuario presiona Enter sin escribir nada, se considera como "Consumidor Final" (C/F).

D. Verificación de entrada para el nombre del cliente

1. Condición:

- Si el nombre del cliente está vacío (isEmpty), asigna el valor "C/F".

2. Entrada del NIT:

- Si se proporciona un nombre, solicita el **NIT** del cliente.
- Utiliza leer.nextInt() para capturar un número entero.
- Llama a leer.nextLine() para limpiar el buffer del scanner.

Registro de productos comprados – Variables iniciales

```
//Registro de productos comprados
String[] productosComprados = new String[100];
int[] cantidadesComprados = new int[100];
double[] preciosComprados = new double[100];
double totalCompra = 0.0;
int productosCompradosCount = 0;

boolean continuarComprando = true;
```

A. Declaración de arreglos

1. productosComprados:

- Almacena los nombres de los productos seleccionados por el cliente.
- Capacidad máxima: 100 productos por venta.

2. cantidadesComprados:

- Guarda las cantidades compradas de cada producto correspondiente al índice en productosComprados.

3. preciosComprados:

- Guarda los precios unitarios de cada producto seleccionado, alineado con los índices de los otros arreglos.

B. Variables auxiliares

1. **totalCompra:**

- Acumula el costo total de los productos seleccionados por el cliente.

2. **productosCompradosCount:**

- Lleva un conteo de los productos añadidos al carrito de compra.
- También actúa como índice para agregar nuevos productos en los arreglos.

C. Control del flujo de compra

- **Propósito:** Define si el cliente desea seguir añadiendo productos al carrito.
- **Valor inicial:** true para entrar al ciclo de selección de productos.
- **Uso futuro:**
 - Se actualizará a false cuando el cliente decida finalizar la compra.

3. Ventajas de la estructura

1. **Organización de datos:**

- Los arreglos paralelos
(productosComprados, cantidadesComprados, preciosComprados)
permiten almacenar y gestionar datos relacionados de manera consistente.

2. **Capacidad fija:**

- Limitar a 100 productos por venta asegura un manejo simple del inventario, aunque puede ser mejorado para escenarios dinámicos.

3. **Preparación para cálculos:**

- totalCompra facilita el cálculo acumulativo del costo total.
- productosCompradosCount evita iterar sobre elementos vacíos.

Selección de productos

```
while(continuarComprando){
    //Mostrar inventario
    System.out.println("Productos disponibles:");
    System.out.println("-----");
    for(int i =0; i < cantidadProducto; i++){
        if(nombreProducto[i] != null){
            System.out.printf("%d. %s - Precio: Q%.2f%n", i + 1, nombreProducto[i], precioProducto[i]);
        }
    }

    //Elegir producto a comprar
    System.out.println("Selecciona el número del producto que deseas comprar");
    int opcionProducto = leer.nextInt() -1; //Por la posición que tiene en el array
    leer.nextLine(); //Limpiar buffer

    //Evitar que el cliente elija una opción que no está || numeros negativos
    if(opcionProducto < 0 || opcionProducto >= cantidadProducto || nombreProducto[opcionProducto] == null){
        System.out.println("Opción invalida, inténtalo de nuevo");
        continue;
    }
}
```

A. Mostrar el inventario

1. Iterar sobre el inventario:

- El ciclo for recorre los productos registrados en el inventario, limitado por cantidadProducto.

2. Validación de productos:

- Verifica que nombreProducto[i] no sea null antes de mostrarlo. Esto asegura que no se impriman posiciones vacías.

3. Formato de salida:

- Muestra el índice del producto (ajustado para que empiece en 1), el nombre, y su precio.

B. Selección del producto

1. Solicitar la selección:

- El cliente ingresa el número del producto que desea comprar.
- El índice ingresado se ajusta restando 1 para alinearse con las posiciones del arreglo (que comienzan en 0).

2. Limpiar el buffer:

- `leer.nextLine()` elimina cualquier dato residual en el buffer del scanner, evitando conflictos en futuras entradas.

C. Validación de la opción seleccionada

1. Condiciones de error:

- `opcionProducto < 0`: Verifica que no se seleccionen índices negativos.
- `opcionProducto >= cantidadProducto`: Asegura que la selección esté dentro del rango del inventario.
- `nombreProducto[opcionProducto] == null`: Evita seleccionar una posición vacía en el arreglo.

2. Acción ante error:

- Muestra un mensaje indicando que la opción es inválida.
- Usa `continue` para reiniciar el ciclo, permitiendo al cliente intentarlo nuevamente.

Registro de cantidad y flujo de compra

```
//Ingresar cantidad deseada
System.out.println("Ingresa la cantidad que deseas: ");
int cantidadDeseada = leer.nextInt();
leer.nextLine(); //Limpiar buffer

//Registrar la compra
productosComprados[productosCompradosCount] = nombreProducto[opcionProducto];
cantidadesComprados[productosCompradosCount] = cantidadDeseada;
preciosComprados[productosCompradosCount] = precioProducto[opcionProducto]; // Registrar precio unitario
totalCompra += precioProducto[opcionProducto] * cantidadDeseada;
productosCompradosCount++;

//Actualizar las ventas del producto
ventasProducto[opcionProducto] += cantidadDeseada;

//Preguntar si desea seguir comprando
System.out.println("Quieres añadir otro producto? (Y/N): ");
String continuar = leer.nextLine();
if(continuar.equalsIgnoreCase("N")){
    continuarComprando = false;
}
}
```

2. Estructura del código

A. Captura de la cantidad deseada

1. Solicitar cantidad:

- El cliente ingresa la cantidad deseada del producto seleccionado.

2. Limpiar el buffer:

- Usa `leer.nextLine()` para evitar problemas en futuras entradas debido al carácter de salto de línea residual.

B. Registrar la compra

1. Agregar producto al carrito:

- Almacena el nombre, cantidad, y precio unitario en los arreglos correspondientes:
 - `productosComprados`: Nombre del producto.
 - `cantidadesComprados`: Cantidad comprada.
 - `preciosComprados`: Precio unitario.

2. Actualizar el total de la compra:

- Calcula el subtotal del producto seleccionado ($\text{precio} * \text{cantidad}$) y lo suma a `totalCompra`.

3. Incrementar el contador de productos comprados:

- `productosCompradosCount` asegura que las próximas entradas se registren en la siguiente posición disponible de los arreglos.

C. Actualizar las ventas del producto

- **Propósito:** Incrementa la cantidad vendida del producto seleccionado en el arreglo `ventasProducto`, que lleva un registro global de las ventas por producto.

D. Preguntar si desea continuar

1. Control del flujo:

- Solicita al cliente que indique si desea agregar otro producto al carrito.

2. Finalización de la compra:

- Si el cliente ingresa "N" (no), la variable continuarComprando se establece como false, saliendo del ciclo while.

Resumen de compra

```
//Mostrar el resumen de compra
System.out.println("\n--- Resumen de la Compra ---");
System.out.println("-----");
System.out.printf("Cliente: %s (NIT: %s)%n", nombreCliente, nitCliente);

for (int i = 0; i < productosCompradosCount; i++) {
    double subtotal = preciosComprados[i] * cantidadesComprados[i]; // Calcular subtotal
    System.out.printf("%d. %s - Cantidad: %d - Precio: Q%.2f - Subtotal: Q%.2f%n",
        i + 1, productosComprados[i], cantidadesComprados[i], preciosComprados[i], subtotal);
}

System.out.println("-----");
System.out.printf("Total de la compra: Q%.2f%n", totalCompra);
System.out.println("-----");
System.out.println("Presiona cualquier tecla para ver tu factura...");
Leer.nextLine();

//YA TENEMOS EL RECIBO DE COMPRA, TOCA HACER EL HTML
generarFactura(nombreCliente, nitCliente, productosComprados, cantidadesComprados, preciosComprados, totalCompra, productosCompradosCount);
numeroVenta++;
}
```

A. Encabezado del resumen

1. Encabezado del cliente:

- Muestra el nombre del cliente y su NIT.
- Si el cliente es "C/F" (consumidor final), el NIT será 0 o un valor equivalente.

B. Detalle de productos comprados

1. Iteración sobre los productos comprados:

- Recorre los arreglos productosComprados, cantidadesComprados, y preciosComprados hasta el valor de productosCompradosCount.

2. Cálculo del subtotal:

- Multiplica la cantidad comprada por el precio unitario para obtener el subtotal de cada producto.

3. Formato del detalle:

- Imprime el índice del producto, su nombre, cantidad comprada, precio unitario, y subtotal.

C. Total de la compra

1. Línea separadora:

- Mejora la legibilidad del resumen.

2. Mostrar el total acumulado:

- Imprime el valor de totalCompra, que ya contiene la suma de todos los subtotales calculados durante la selección de productos.

3. Espera del usuario:

- Solicita al usuario presionar cualquier tecla antes de generar la factura, permitiendo revisar el resumen antes de continuar.

D. Generación de la factura

1. Llamada a generarFactura:

- Envía todos los datos recopilados (cliente, productos, total, etc.) al método generarFactura, encargado de crear el archivo HTML.

2. Incremento del número de venta:

- Actualiza el contador numeroVenta para garantizar que la próxima factura tenga un identificador único.

Generación de factura (Directorio y estructura)

```
//OPERACIÓN PARA GENERAR LA FACTURA
public static void generarFactura(String cliente, int nit, String[] productos, int[] cantidades, double[] precios, double total, int productosCount){
    //DEFINIENDO EL DIRECTORIO DE LAS FACTURAS
    String directorio = "src/Facturas";
    File carpetaFacturas = new File(directorio);

    //CODIGO CREADO POR RICHARD ARIZANDIETA, LA COPIA PARCIAL O TOTAL DE ESTE CODIGO SERÁ REPORTADA CON LAS AUTORIDADES CORRESPONDIENTES CON RESPECTIVA

    // Verificar si la carpeta existe
    if (!carpetaFacturas.exists()) {
        if (carpetaFacturas.mkdirs()) {
            System.out.println("Carpeta 'Facturas' creada exitosamente.");
        } else {
            System.out.println("Error al crear la carpeta 'Facturas'.");
            return;
        }
    }
}
```

A. Definición del directorio

1. Directorio de facturas:

- Define la ruta donde se almacenarán los archivos generados.
- "src/Facturas" representa una ubicación relativa dentro del proyecto.

2. Objeto File:

- Utiliza File para representar el directorio de facturas y realizar operaciones como verificar si existe o crearlo.

B. Verificación y creación del directorio

1. Verificar existencia del directorio:

- carpetaFacturas.exists():
 - Retorna true si el directorio ya existe.
 - Si no existe, entra en el bloque if para intentar crearlo.

2. Crear el directorio:

- carpetaFacturas.mkdirs():
 - Crea la carpeta especificada en directorio.
 - Si la carpeta se crea con éxito, muestra un mensaje confirmando la operación.

3. Manejo de errores:

- Si la carpeta no puede ser creada, muestra un mensaje de error y termina la ejecución del método con return.

Generación del archivo HTML de la factura

```
//RUTA COMPLETA DEL ARCHIVO
String nombreArchivo = directorio + "/Factura no. (" + numeroVenta + ").html";

try(PrintWriter writer = new PrintWriter(nombreArchivo)){
    writer.println("<!DOCTYPE html>");
    writer.println("<html>");
    writer.println("<head>");
        writer.println("<title>Factura de Compra</title>");
        writer.println("<link rel='stylesheet' href='../style.css'>");
    writer.println("</head>");
    writer.println("<body>");
    writer.println("<h1> Resumen de compra </h1>");

    //COMPROBANDO SI SE ELIGIÓ CF O NOMBRE
    if(cliente.equalsIgnoreCase( anotherString: "C/F")){
        writer.println("<p>Nombre: Consumidor Final </p>");
        writer.println("<p>NIT: Consumidor Final </p>");
    }else {
        writer.println("<p>Nombre de cliente: " + cliente + "</p>");
        writer.println("<p>NIT: " + nit + "</p>");
    }

    writer.println("<h2> Productos comprados: </h2>");
}
```

A. Ruta del archivo

1. Nombre del archivo:

- Combina el directorio definido previamente con un nombre único para la factura.
- El identificador numeroVenta asegura que cada factura tenga un nombre distinto.

2. Extensión HTML:

- Indica que el archivo será un documento HTML, permitiendo abrirlo en un navegador.

B. Manejo del archivo con PrintWriter

- **PrintWriter:** Permite escribir texto en el archivo de manera sencilla.
- **try con recursos:** Garantiza que el archivo se cierre automáticamente después de su uso.

C. Estructura básica del HTML

1. Declaración del DOCTYPE:

- Indica que el documento es HTML5.

2. Etiquetas principales:

- `<html>`: Marca el inicio del documento HTML.
- `<head>`: Contiene metadatos, como el título y la hoja de estilos.
- `<body>`: Contiene el contenido visible del documento.

3. Título:

- Define el título de la factura como "Factura de Compra".

4. Hoja de estilos:

- Vincula un archivo CSS (`../style.css`) para personalizar el diseño de la factura.

5. Encabezado principal:

- Muestra un título en el cuerpo del documento: "Resumen de compra".

D. Información del cliente

1. Consumidor final:

- Si el cliente es "C/F", muestra un texto genérico para el nombre y el NIT.

2. Cliente con nombre y NIT:

- Muestra el nombre del cliente y su NIT tal como fueron capturados.

E. Encabezado de productos

- **Título:** Introduce la sección que listará los productos comprados.
- **Estructura de la tabla:** La etiqueta `<tbody>` marca el inicio del cuerpo de una tabla, donde se incluirán los productos.

Tabla de productos

```
// Iniciar la tabla
writer.println("<table>");
writer.println("<thead>");
writer.println("<tr>");
writer.println("<th>#</th>");
writer.println("<th>Producto</th>");
writer.println("<th>Cantidad</th>");
writer.println("<th>Precio Unitario</th>");
writer.println("<th>Subtotal</th>");
writer.println("</tr>");
writer.println("</thead>");
writer.println("<tbody>");

// Ciclo para añadir filas con los productos
for (int i = 0; i < productosCount; i++) {
    double subtotal = cantidades[i] * precios[i]; // Calcular subtotal
    writer.println("<tr>");
    writer.printf("<td>%d</td>%n", i + 1); // Número del producto
    writer.printf("<td>%s</td>%n", productos[i]); // Nombre del producto
    writer.printf("<td>%d</td>%n", cantidades[i]); // Cantidad
    writer.printf("<td>Q%.2f</td>%n", precios[i]); // Precio unitario
    writer.printf("<td>Q%.2f</td>%n", subtotal); // Subtotal
    writer.println("</tr>");
}

writer.println("</tbody>");
writer.println("</table>");
```

A. Iniciar la tabla

1. <table>:

- Marca el inicio de una tabla en HTML.

2. Encabezado de la tabla (<thead>):

- <tr>: Representa una fila de la tabla.
- <th>: Define las celdas de encabezado para cada columna:
 - #: Número del producto.
 - Producto: Nombre del producto comprado.
 - Cantidad: Cantidad comprada.
 - Precio Unitario: Precio por unidad del producto.
 - Subtotal: Resultado de Cantidad × Precio.

3. Cuerpo de la tabla (<tbody>):

- Contendrá las filas con los productos específicos seleccionados por el cliente.

B. Agregar productos a la tabla

1. Cálculo del subtotal:

- Multiplica la cantidad comprada por el precio unitario para obtener el costo total de cada producto.

2. Filas dinámicas (<tr>):

- Genera una fila por cada producto en productos.

3. Celdas de datos (<td>):

- i + 1: Número del producto (1-indexed).
- productos[i]: Nombre del producto.
- cantidades[i]: Cantidad comprada.
- precios[i]: Precio unitario.
- subtotal: Costo total para esa línea.

4. Formato con printf:

- Utiliza el formato Q%.2f para asegurar que los precios y subtotales se muestren con dos decimales, manteniendo un formato legible para monedas.

C. Finalizar la tabla

1. Cerrar el cuerpo de la tabla (</tbody>):

- Marca el final de las filas con datos dinámicos.

2. Cerrar la tabla (</table>):

- Finaliza la estructura de la tabla en HTML.

Finalización de la factura

```
        writer.printf("<h2> Total de la compra </h2>");
        writer.printf("Total a pagar: Q%.2f%n", total);
        writer.println("<br>");
        writer.println("<br>");
        writer.println("<b> Muchas gracias por su compra! </b>");
        writer.println("</body>");
        writer.println("</html>");
    } catch (IOException e) {
        System.out.println("Error al crear el archivo: " + e.getMessage());
    }
}
```

A. Mostrar el total de la compra

1. Encabezado del total:

- Utiliza <h2> para destacar la sección del total de la compra.

2. Mostrar el total:

- Imprime el valor de total, formateado con dos decimales (Q%.2f), para garantizar un formato estándar de moneda.

B. Mensaje de agradecimiento

1. Salto de línea:

- Los `<br>` agregan espacios visuales antes del mensaje de agradecimiento.

2. Mensaje de cierre:

- Utiliza `<b>` para resaltar el mensaje de agradecimiento en negrita.

C. Finalizar el HTML

1. Cerrar `<body>`:

- Indica el fin del contenido principal del documento HTML.

2. Cerrar `<html>`:

- Marca el final del archivo HTML.

D. Manejo de excepciones

1. Captura de errores:

- Si ocurre un error al crear o escribir en el archivo, se captura una excepción `IOException`.

2. Mensaje de error:

- Muestra en consola un mensaje indicando la causa del error, proporcionada por `e.getMessage()`.

Realizar un reporte de ventas

```
//OPERACIÓN PARA REALIZAR UN REPORTE DE VENTAS
//VARIABLE PARA CONTAR LAS VENTAS DE CADA PRODUCTO
static int[] ventasProducto = new int[100]; 3 usages

//OPERACIÓN PARA REALIZAR UN REPORTE
public static void realizarReporte(Scanner leer){ 1 usage
    System.out.println("Realizar un reporte");
    System.out.println("-----");
    System.out.println("1. Top 5 de productos más vendidos");
    System.out.println("2. Reporte total de ventas");
    System.out.println("Elije una opción para continuar:");
    int opcion = leer.nextInt();
    leer.nextLine(); //Limpiar buffer

    switch(opcion){
        case 1:
            topProductos(leer);
            break;
        case 2:
            reporteTotal(leer);
            break;
        default:
            System.out.println("Opción invalida, intenta de nuevo");
    }
}
```

A. Variable para las ventas

1. ventasProducto:

- Arreglo que lleva el conteo de las unidades vendidas de cada producto.
- **Índices:** Coinciden con los productos registrados en los arreglos nombreProducto y precioProducto.

B. Método realizarReporte

1. Menú de opciones:

- Presenta al usuario dos tipos de reportes:
 - **Opción 1:** Muestra el top 5 de productos más vendidos.
 - **Opción 2:** Presenta un desglose total de las ventas realizadas.

2. Captura de opción:

- Recoge la entrada del usuario para determinar qué reporte generar.
- Usa `leer.nextLine()` para limpiar el buffer después de leer el número.

C. Lógica del menú

1. case 1:

- Llama al método `topProductos`, que genera un reporte con los cinco productos más vendidos.

2. case 2:

- Llama al método `reporteTotal`, que muestra un desglose completo de las ventas de cada producto.

3. Default:

- Maneja entradas no válidas mostrando un mensaje de error.

Top 5 productos más vendidos

```
//FUNCIÓN DE TOP 5 PRODUCTOS
public static void topProductos(Scanner leer){ 1 usage
    //Copias de los arrays de los productos vendidos
    String[] copiaProductos = nombreProducto.clone();
    int[] copiaVentas = ventasProducto.clone();

    //Ordenar los productos por ventas de mayor a menor
    for (int i = 0; i < copiaVentas.length - 1; i++){
        for(int j = 0; j < copiaVentas.length - i - 1; j++){
            if(copiaVentas[j] < copiaVentas[j + 1]){
                //Vamos reordenando las ventas
                int tempVentas = copiaVentas[j];
                copiaVentas[j] = copiaVentas[j + 1];
                copiaVentas[j + 1] = tempVentas;

                //Ahora reordenamos los productos
                String tempProductos = copiaProductos[j];
                copiaProductos[j] = copiaProductos[j + 1];
                copiaProductos[j + 1] = tempProductos;
            }
        }
    }
}
```

A. Clonar los arreglos

1. Propósito de las copias:

- Evita modificar directamente los arreglos originales nombreProducto y ventasProducto, manteniendo su integridad.
- Permite realizar el ordenamiento en los arreglos clonados.

2. Uso de .clone():

- Crea una copia independiente de los arreglos originales.

B. Ordenamiento de los productos

1. Método de ordenamiento:

- Implementa el algoritmo de **ordenamiento burbuja**.
- Compara pares consecutivos de elementos en copiaVentas y los intercambia si están en el orden incorrecto (menor antes que mayor).

2. Sincronización de arreglos:

- Cuando se intercambian los valores en copiaVentas, también se intercambian los valores correspondientes en copiaProductos para mantener la relación entre las ventas y sus productos.

3. Condición de ordenamiento:

- copiaVentas[j] < copiaVentas[j + 1]:
 - Ordena en orden descendente, de mayor a menor.

Mostrar el Top 5 de productos más vendidos

```
//MOSTRANDO EL TOP 5
System.out.println("\n--- Top 5 de productos más vendidos ---");
System.out.println("-----");

for(int i = 0; i < 5 && i < copiaProductos.length && copiaProductos[i] != null; i++) {
    System.out.printf("%d. %s - Cantidad Vendida: %d\n", i + 1, copiaProductos[i], copiaVentas[i]);
}

System.out.println("-----");
System.out.println("Presiona cualquier tecla para continuar...");
leer.nextLine();
}
```

A. Encabezado del reporte

1. Encabezado descriptivo:

- Introduce la sección del reporte para que el usuario sepa que se trata de los productos más vendidos.

2. Separadores:

- Ayudan a organizar visualmente la información en consola.

B. Iteración sobre los productos

1. Condiciones del bucle:

- `i < 5`: Limita el reporte a los cinco productos más vendidos.
- `i < copiaProductos.length`: Garantiza que el bucle no intente acceder a índices fuera de rango.
- `copiaProductos[i] != null`: Evita mostrar celdas vacías en caso de que haya espacios nulos en el arreglo.

2. Contenido del reporte:

- `i + 1`: Número del producto en el top, comenzando desde 1.
- `copiaProductos[i]`: Nombre del producto.
- `copiaVentas[i]`: Cantidad vendida de ese producto.

3. Formato de salida:

- Usa `printf` para presentar la información en una línea, con formato amigable para el usuario.

C. Finalización del reporte

1. Separador final:

- Indica que el reporte ha terminado, mejorando la presentación.

2. Pausa para el usuario:

- Usa `leer.nextLine()` para permitir al usuario revisar el reporte antes de regresar al menú principal.

Reporte total de ventas

```
//FUNCION DE REPORTE TOTAL DE PRODUCTOS
public static void reporteTotal(Scanner leer){ 1 usage
    System.out.println("\n--- Reporte Total de Ventas ---");
    System.out.println("-----");
    for(int i = 0; i < cantidadProducto; i++){
        if(nombreProducto[i] != null){
            System.out.printf("%d. %s - Cantidad Vendida: %d\n", i + 1, nombreProducto[i], ventasProducto[i]);
        }
    }
    System.out.println("-----");
    System.out.println("Presiona cualquier tecla para continuar...");
    leer.nextLine();
}
```

A. Encabezado del reporte

1. Título del reporte:

- Informa al usuario que se generará un desglose completo de las ventas.

2. Separadores:

- Ayudan a organizar visualmente la información.

B. Iteración sobre los productos

1. Recorrido del inventario:

- El bucle for itera hasta cantidadProducto, asegurándose de procesar solo las posiciones ocupadas del arreglo.

2. Verificación de productos nulos:

- Evita imprimir productos no registrados verificando que nombreProducto[i] no sea null.

3. Formato de salida:

- i + 1: Muestra el número del producto, comenzando desde 1.
- nombreProducto[i]: Nombre del producto.
- ventasProducto[i]: Cantidad total vendida del producto.

C. Finalización del reporte

1. Separador final:

- Indica que el reporte ha terminado.

2. Interacción pausada:

- Usa `leer.nextLine()` para permitir al usuario revisar el reporte antes de regresar al menú principal.